Høgskulen
på Vestlandet




# DAT108 - Programmering og webapplikasjoner




## F16 Webapps med Spring Boot




Lars-Petter Helland
Bergen
9. oktober 2026

# Plan for dagen

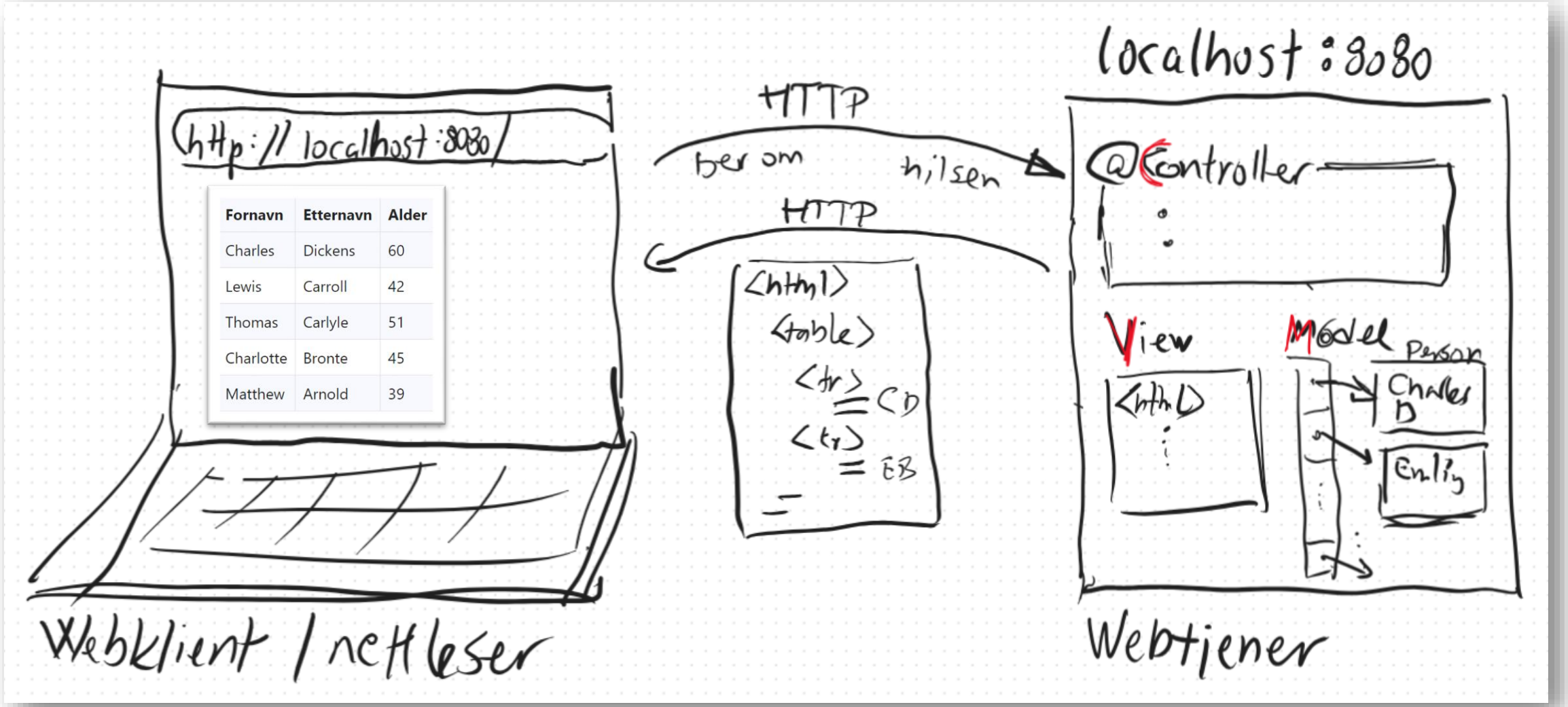
- Repetisjon fra sist. HTML-side generert på webtjeneren, `/personliste`

- Hva er Spring, Spring MVC og Spring Boot
- Hva vil det si å "kjøre" en Spring Boot web-applikasjon
- Hva er Maven. Hvordan opprette et nytt Spring Boot-prosjekt

- Se litt nærmere på en Spring MVC @**Controller**.
- Se litt nærmere på et Spring MVC **Model**-objekt.
- Se litt nærmere på et Thymeleaf **View**.

- Et eksempel vi gjør sammen (bokliste).

# Recap fra sist: HTML-side generert på webtjeneren, `/personliste`



# Dagens lesestoff

- Spring MVC og Spring Boot
  - Les gjerne på egen hånd. Kun gjennomgått stoff (forelesninger/eksempler/oppgaver) er pensum.
- Maven
  - Ikke pensum direkte.
- Thymeleaf
  - Har laget og lagt ut et Thymeleaf CheatSheet for DAT108.pdf som gir en grei oversikt
  - Les også: ...

# Spring, Spring MVC og Spring Boot

*"Hva er Spring, Spring MVC og Spring Boot, og hvordan henger disse tingene sammen?"*

- **Spring-rammeverket**

  "Spring er et omfattende rammeverk for applikasjonsutvikling i Java. ..."
- **Spring MVC**

  "Spring MVC (Model-View-Controller) er en modul i Spring-rammeverket som er spesielt designet for å bygge webapplikasjoner. ..."
- **Spring Boot**

  "Spring Boot er et prosjekt i Spring-økosystemet som har til hensikt å gjøre utvikling av Spring-applikasjoner enklere og raskere. Det er spesielt utformet for å gjøre oppsett og konfigurasjon av nye prosjekter smidig og lite arbeidskrevende. ... Gir mulighet til å kjøre applikasjoner uten behov for å sette opp en ekstern server som Tomcat eller Jetty."

# Å "kjøre" en Spring Boot web-applikasjon

- En av fordelene med å bruke Spring Boot, er at vi kan "bake inn" (embed) en webserver i applikasjonen vår, og at vi derfor kan kjøre applikasjonen (inkl. webserveren) som en standard Java-applikasjon. Dette er nyttig under utvikling for kjøring på egen PC.

- Å "kjøre" main-metoden i et Spring-prosjekt betyr egentlig **å kjøre webserveren**, og at denne har registrert og håndterer vår applikasjon.

- Når applikasjonen er ferdig og testet ut på egen maskin er det derimot naturlig at denne "deployes" på en webserver som allerede er oppe og går, som er tilgjengelig på internett, og kanskje også har et registrert domene.

  (Dette kommer vi tilbake til senere i kurset !)

# Maven

En Spring MVC web-applikasjon er bygget opp av kode fra minst et 10-talls ulike kodebiblioteker (jar-filer) i ulike versjoner. Hvert kodebibliotek er kanskje avhengig av andre kodebibliotek i visse versjoner, ...

Vi trenger et verktøy for å håndtere alle direkte og transitive avhengigheter til kodebiblioteker i prosjektene våre.

I Java-prosjekter er det to standard-verktøy for dette. Det ene er Gradle, og det andre er **Maven**.

Hva gjør Maven for oss:

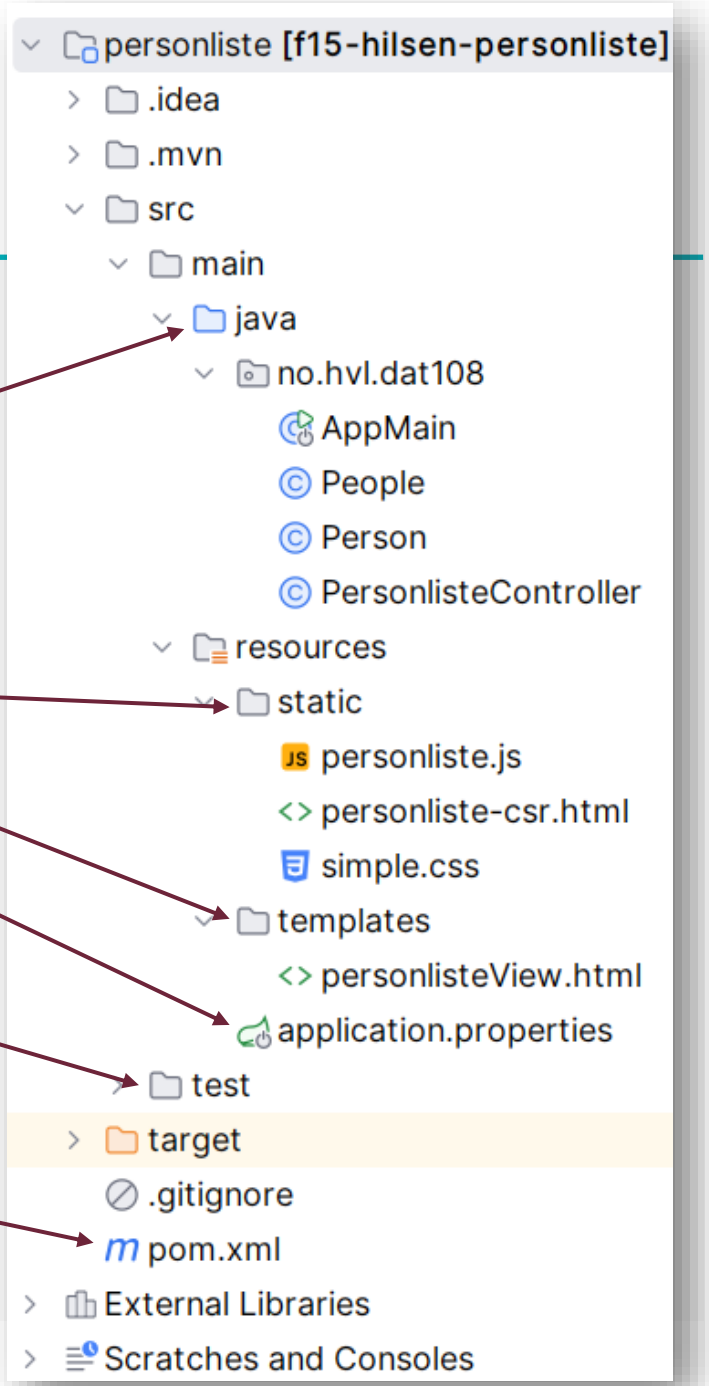
- Gir oss **et fast prosjektoppsett** uavhengig av IDE
- Lar oss konfigurere prosjektet via én enkelt fil kalt **pom.xml**
- Håndterer automatisk avhengigheter til kodebibliotek
- Lar oss automatisere byggeprosessen, slik at prosjektet kan bygges uavhengig av den enkelte utviklers oppsett (skal ikke se på dette)

# Prosjektoppsett

Et Java web-prosjekt har en del «ingredienser».

Her er en oversikt over hvor ting er plassert.

- Java-filer ligger under `src/main/java`
- Statiske filer ligger under `src/main/resources/static`
- HTML-maler ligger under `src/main/resources/templates`
- `application.properties` ligger under `src/main/resources`
- Test-filer (f.eks. JUnit-tester) ligger under `src/test/...`
- `pom.xml` ligger i roten av prosjektet



# pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
        <parent>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-parent</artifactId>
          <version>4.1.1</version>
          <relativePath /> <!-- lookup parent from repository -->
        </parent>

        <groupId>no.hvl.dat108.bokliste</groupId>
        <artifactId>f16-bokliste</artifactId>
        <version>0.0.1-SNAPSHOT</version>

        <properties><java.version>25</java.version></properties>

        <dependencies>
          <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
          </dependency>
          <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
          </dependency>
        </dependencies>


    ...
```

# Samspillet mellom Maven og IntelliJ

- **Vi velger å la Maven styre våre Spring Boot-prosjekter** !

- NB! Hvis vi gjør endringer i pom.xml-filen vil Maven-oppsettet og IntelliJ-oppsettet kanskje ikke lenger være i sync. For å oppdatere IntelliJ-prosjektet kan vi gjøre:

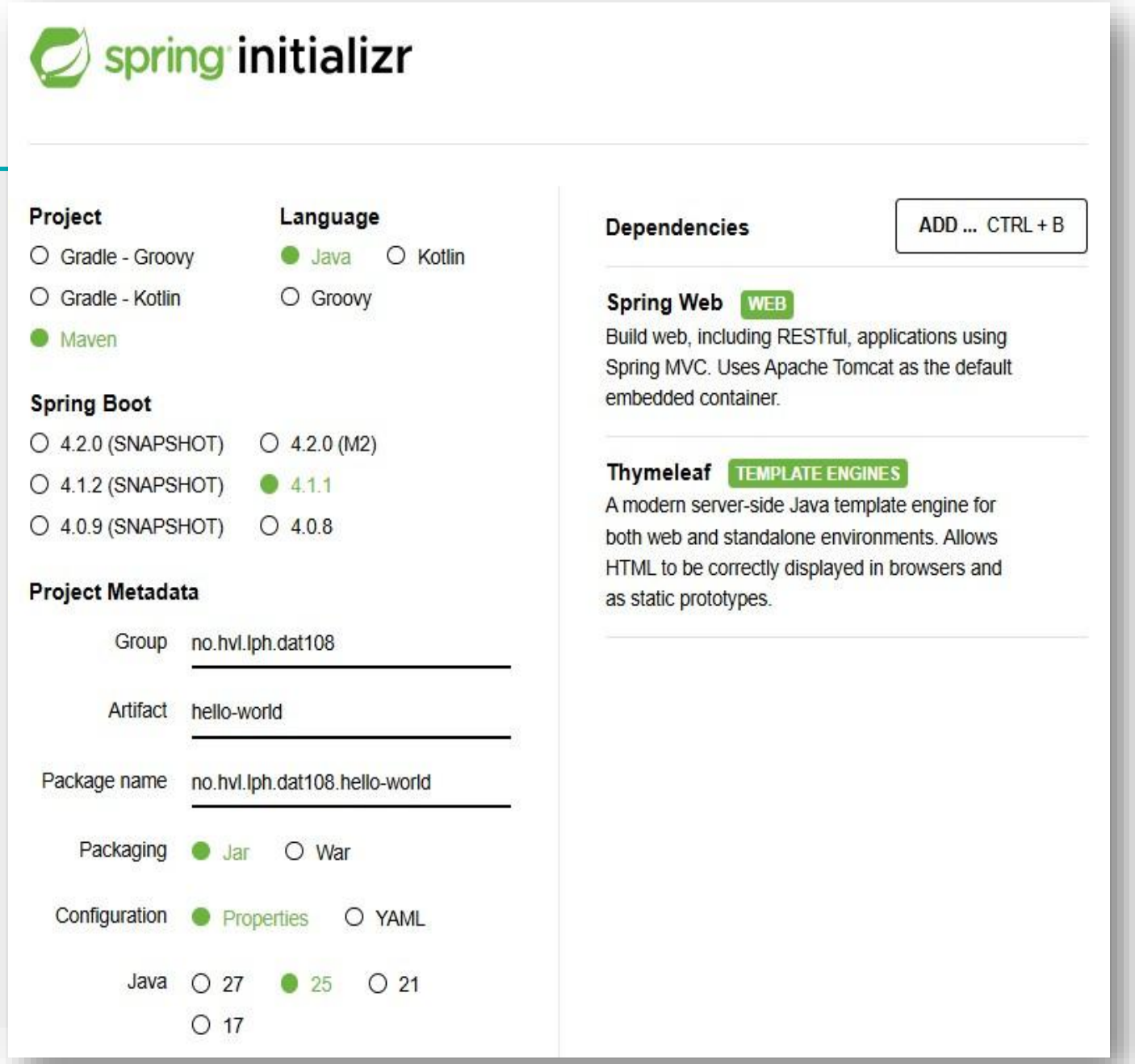
  - Høyreklikk **| Maven |** 🔄 **Sync Project**

# Hvordan opprette et nytt tomt prosjekt

- Alt1: spring initializr https://start.spring.io/ (anbefalt). Får en zip-fil som kan pakkes ut og åpnes direkte i IntelliJ.

**Vi gjør dette sammen nå!**

- Alt2: Plugin for initializr (krever IntelliJ Ultimate sub)
- Alt3: Ta en kopi av et tidligere prosjekt



# application.properties (evt. application.yaml)

- Innstillinger i Spring Boot -applikasjoner gjøres i filen application.properties.
- Det er mange ting som kan gjøres her. Foreløpig:

```
# Her sier vi at applikasjonen har et navn, og at ressurser
# er relativt under denne, f.eks.
#     http://localhost:8080/f15app/...
server.servlet.context-path=/f15app

# Her gir vi beskjed om hvor mye loggedetaljer vi onsker
# Nivaaene er TRACE, DEBUG, INFO, WARN, ERROR
logging.level.root=INFO
logging.level.org.springframework.web=DEBUG
```

- **NB! Ikke bruk norske bokstaver, heller ikke i kommentarer, her !!!!!!!!**

# Nå litt mer om Spring MVC



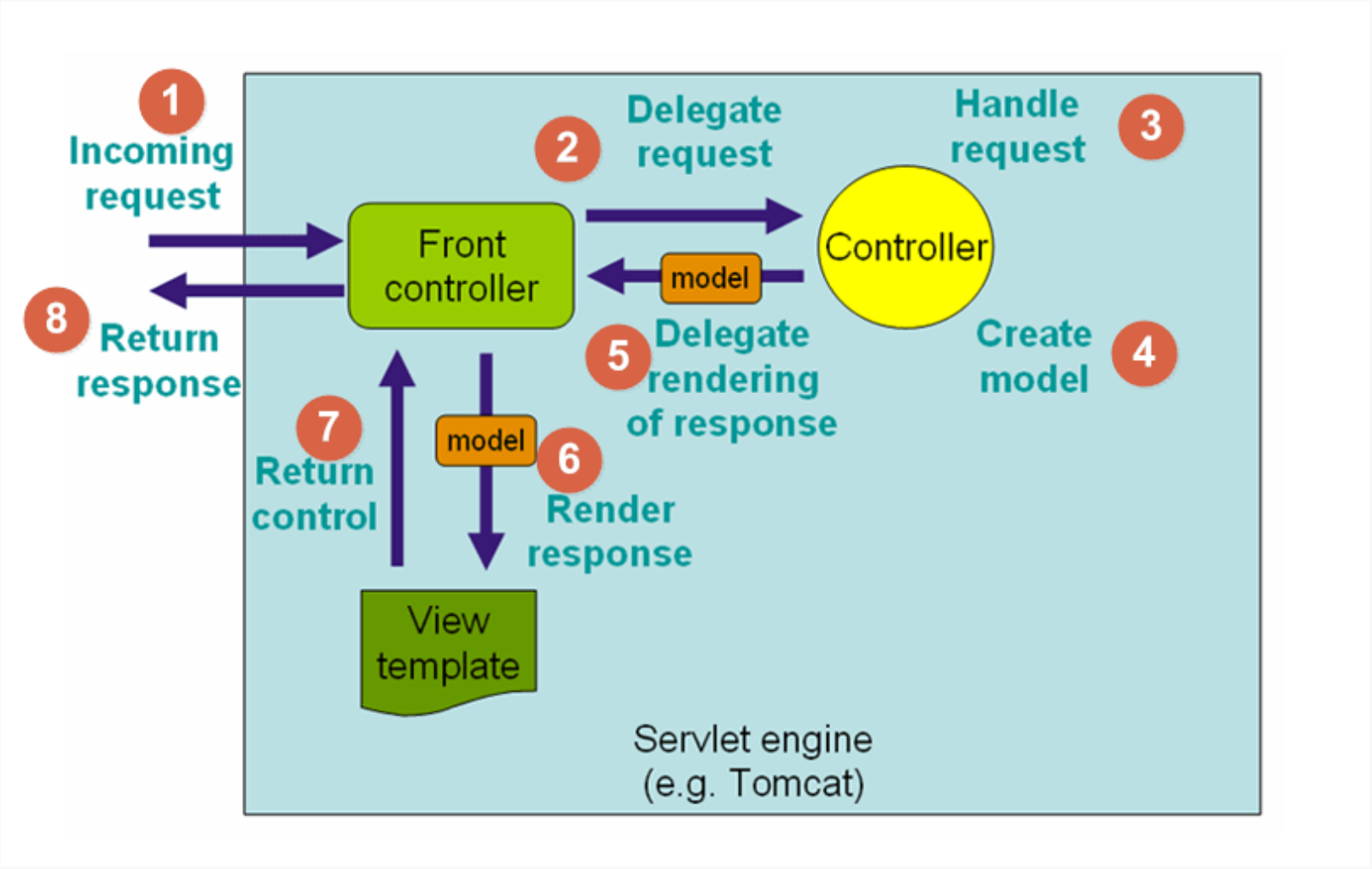
# Hva er Spring MVC

Fra Google AI:

- **Spring MVC** er en modul i Spring Framework som brukes til å bygge webapplikasjoner og REST API-er i Java. [1, 2, 3]
- Rammeverket er bygget på **Model-View-Controller**-mønsteret (MVC), som deler opp applikasjonen i tre hoveddeler for å sikre god struktur og løsere kobling: [1, 2]
- **Model (Modell):** Inneholder applikasjonens data, forretningslogikk og domenteiteter (ofte POJOs – Plain Old Java Objects). [1]
- **View (Visning):** Tar seg av brukergrensesnittet og viser frem data fra modellen (f.eks. HTML/JSP, Thymeleaf eller JSON for REST-tjenester). [1, 2]
- **Controller (Kontroller):** Tar i mot HTTP-forespørsler fra brukeren, kaller logikken i modulen og sender riktig respons eller visning tilbake. [1]

# Spring MVC arbeidsflyt

https://www.onlinetutorialspoint.com/spring/spring-web-mvc-framework.html



# Hva er en Spring MVC @**Controller**

- En Spring MVC @Controller er en klasse som har ansvar for å behandle innkomne forespørsler (requests) for én eller flere stier i applikasjonen.
- I eksemplet under vil en **HTTP GET**-request til stien `/personliste` bli mottatt og behandlet av metoden `personliste(..)`.
- Hvert metodekall til `personliste(..)` blir kjørt i en egen tråd på serveren.

```
@Controller
public class PersonlisteController {

    @GetMapping("/personliste")
    public String personliste(Model model) {
        ...
    }
}
```

- En Spring webapplikasjon kan gjerne ha flere/mange @Controllere.

# Hva er et Spring MVC **Model**-objekt

- Hver request til applikasjonen får tilordnet sitt eget Model-objekt der vi blant annet kan mellomlagre data som trengs senere i requesten.
- Model-objekter inneholdet et Map<String, Object> av attributter som brukes til å ta vare på data.

```
@GetMapping("/personliste")
public String personliste(Model model) {
    model.addAttribute("personliste", People.people);
    ...
}
```

# Hva er et Thymeleaf **View**

- Thymeleaf brukes til å kombinere statisk HTML og dynamiske data for å generere innhold i en respons.
- Thymeleaf (-parseren) blir kjørt på serveren !!
- Returverdien etter kjøring er det ferdige genererte HTML-dokumentet.
- Et par eksempler:

  Å vise en verdien til modell-attributt

  ```
  <p th:text="${melding}" />
  ```
  gir oss `<p>Hei på deg</p>`

  Å iterere over en samling

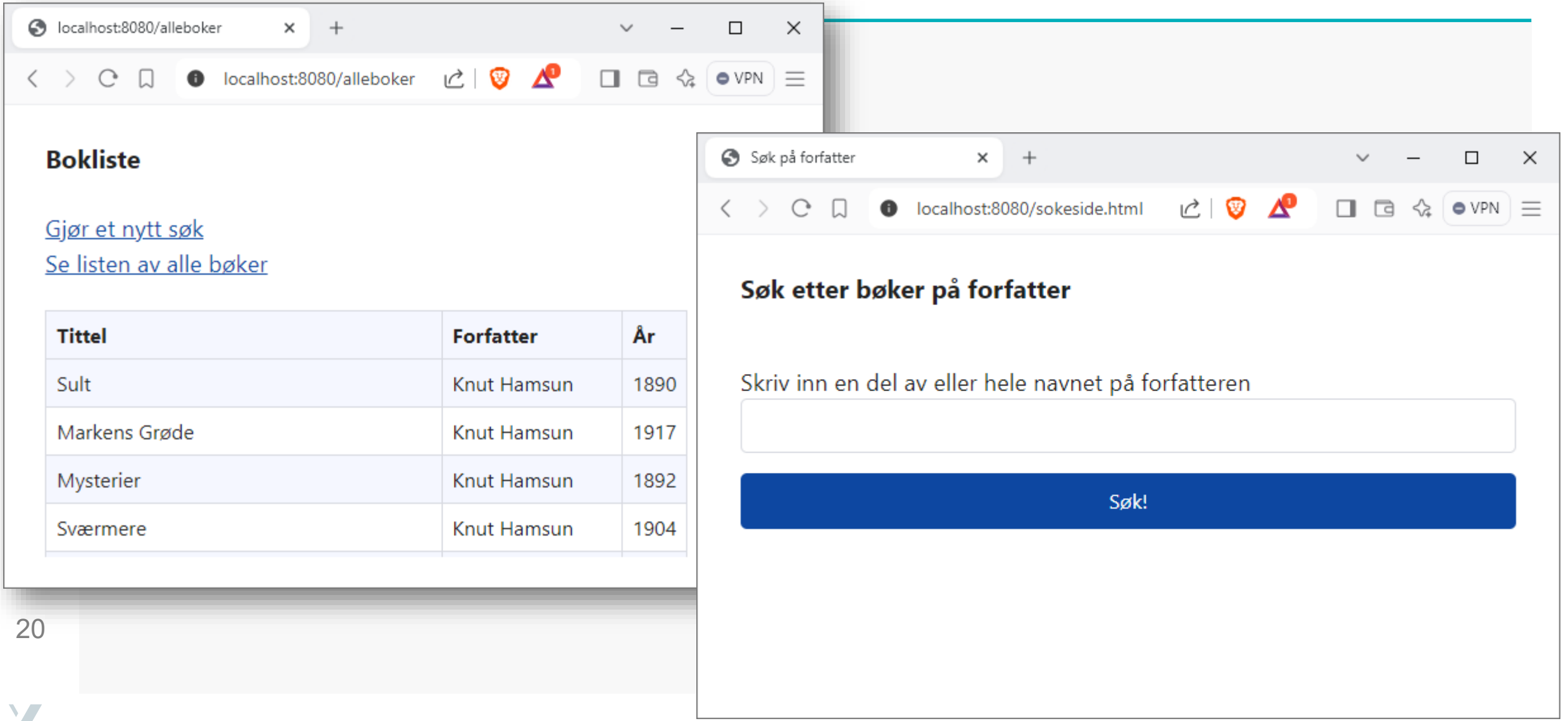
  ```
  <p th:each="m: ${meldinger}" th:text="${m}" />
  ```
  gir oss flere p-tagger

  Å trekke ut objektegenskaper

  ```
  <p th:text="${person.navn}" />
  ```
  gir oss `<p>Arne</p>`


Pause

# Gjøre et prosjekt sammen i timen



# Prosjekt bokliste

Prosjekt: 📂 f16-bokliste

Tar utgangspunkt i fyllinn-prosjektet i GitHub



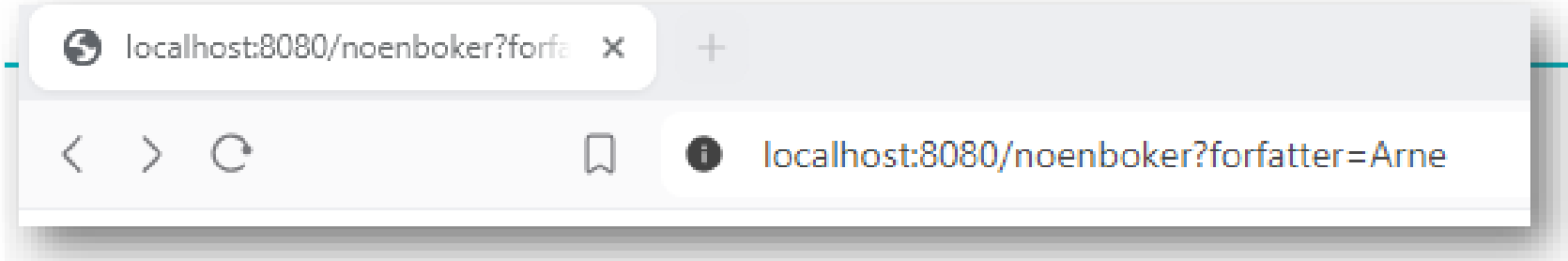
# Fra eksempelet - Basic controller-logikk

- Minimal/basic Controller-logikk er:
  1. Opprett og ta vare på nødvendige **modell**data
  2. Delegér generering av respons til et **view**.

- For eksempel slik:

```
@GetMapping("/alleboker")
public String alleBoker(Model model) {
      model.addAttribute("boker", Boker.alleBoker); //1
      return "boklisteView"; //2
}
```

# Fra eksempelet - HTTP request-parameter




- I eksempelet sender vi med HTTP-parameteren **forfatter** med verdien f.eks. **Arne**
- I controller-metoden får vi tilgang til denne parameteren direkte hvis vi bruker det samme navnet på metode-parameteren , slik:

```
@GetMapping("/noenboker")
public String noenBoker(Model model, String forfatter) { ...
```

- Hvis vi ønsker å tillate at parameteren mangler, kan vi annotere slik:

```
@GetMapping("/noenboker")
public String noenBoker(Model model,
        @RequestParam(required = false) String forfatter) { ...
```

, men vi bør da huske å håndtere tilfellet der den er null:

```
if (forfatter == null ...
```

# Fra eksempelet - Data og logikk i view

- View-et sin oppgave er å generere responsen ut fra allerede ferdiglagde (av controlleren) data.

- Modell-attributter og objekt-egenskaper:

```
${boker}          <!-- Refererer til attributtnavnet "boker" -->
${bok.tittel}     <!-- tittel betyr metodekall til (get)tittel() -->
```

- Innsetting av en tekst, her i en tabell-celle:

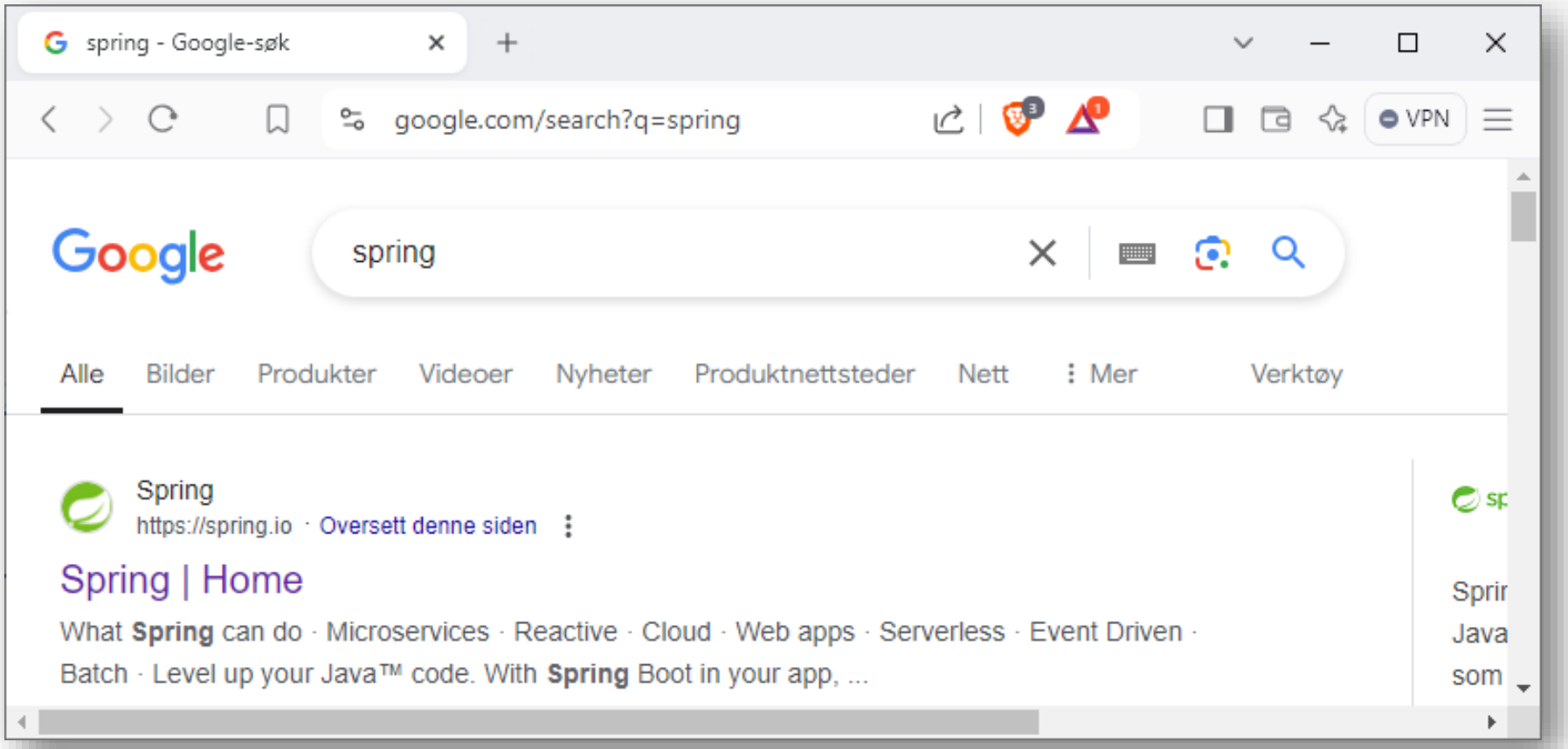
```
<td th:text="${b.forfatter}" />
```

- Løkker, her hvis vi ønsker et antall tabell-rader:

```
<tr th:each="bok : ${boker}"> ... </tr>
```

# Fra eksempelet - Input-skjema

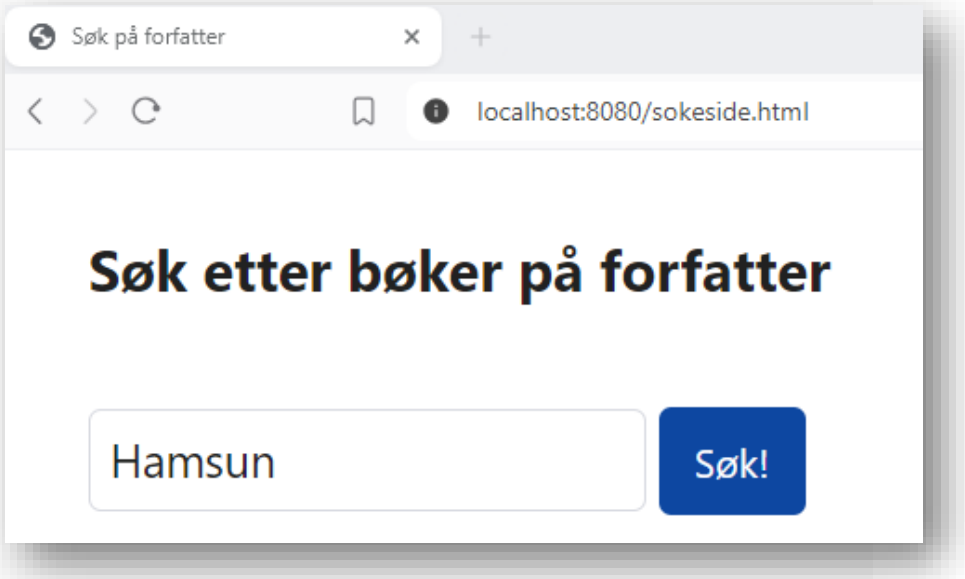
- I stedet for å måtte skrive URL inkl. parametre for hånd i adressefeltet i nettleseren, er det jo bedre å taste inn data i et skjema og trykke på "submit".



# Fra eksempelet - HTML <form> og <input>

- Man lager et slikt skjema i HTML ved å bruke taggene <form> og <input>, f.eks. slik:

```
<form action="noenboker" method="get">
    <input type="text" name="forfatter">
    <input type="submit" value="Søk!">
</form>
```




- Når man "sender inn" skjemaet, enten ved å taste Enter eller trykke på knappen, vil dette resultere i følgende HTTP **GET**-request:

  http://.../**noenboker**?**forfatter**=Hamsun

# Videre

Ingen oppgaver i helgen, men viktig å få eksempler fra forelesninger til å kjøre på egen PC !!

Full fart fra neste uke. Da kommer både frivillige oppgaver og Oblig 4.

Neste gang (mandag):

- Vi ser nærmere på HTTP. Hva er GET og hva er POST
- HTTP Headere og returkoder
- HTML for å lage et skjema der bruker kan gi inn data
- Alle de ulike input-typene i et slikt skjema
- @Controller som tar imot disse dataene, inkl. enkel validering av input